DevOps & Development Workflow

> **Purpose:** End-to-end guide covering planning → design → development → deployment → monitoring. Use this as a living reference for every project.

🗺️ Workflow at a Glance

```
Plan → UI/UX Design → Team Discussion → Task Division
  → Development → Unit Testing → Build → Testing
  → Pipeline Setup → Deployment → Platform Engineering
  → Monitoring & Observability
```

📋 Phase 1 — Planning & Design

> 🎯 *Define the "why" before writing a single line of code.*

Step	Description
📌 **Planning**	Define project requirements, goals, and scope
🎨 **UI/UX Design**	Create mockups and user interface specifications
🤝 **Stakeholder Discussion**	Review with dev team lead and key stakeholders

Checklist

- [ ] Requirements document signed off
- [ ] Wireframes / mockups approved
- [ ] Scope locked; change process defined
- [ ] Stakeholder sign-off received

👥 Phase 2 — Team Organisation

> 🧩 *Who does what, and by when.*

Step	Description
🗂️ **Task Division**	Break work into manageable, trackable tasks
📅 **Timeline Definition**	Assign deadlines and milestones
👩‍💻 **Resource Allocation**	Distribute tasks across team members

Recommended Tools

Tool	Best For
Jira	Issue tracking, sprint planning, engineering workflows
Asana	Task management, cross-team collaboration

Checklist

- [ ] Tasks created in Jira / Asana
- [ ] Milestones and deadlines set
- [ ] Each task has an owner
- [ ] Dependencies mapped

💻 Phase 3 — Development Stage

> 🛠️ *Build with quality baked in from day one.*

`Tool Selection → Development → Unit Testing → Code Review`

Step	Description
🔧 **Tool Selection**	Choose frameworks, languages, and dev tooling
✍️ **Development**	Implement code based on approved specifications
🧪 **Unit Testing**	Write and run unit tests alongside development
🔍 **Code Review**	Review for quality, standards, and security

Checklist

- [ ] Tech stack agreed and documented
- [ ] Branching strategy defined (e.g. Git Flow / trunk-based)
- [ ] Unit test coverage ≥ agreed threshold

- [ ] All PRs reviewed before merge
- [ ] Linting and formatting rules enforced

🏗️ Phase 4 — Build & Testing

> ✅ *Nothing ships without passing the gate.*

`Build Prep → Build Generation → Integration Testing → System Testing → UAT`

Step	Description
📦 **Build Preparation**	Prepare code and configs for production build
⚙️ **Build Generation**	Create build artifacts (Docker images, bundles, etc.)
🧬 **Testing**	Integration → System → Acceptance testing

Testing Layers

Layer	What it Checks
Unit	Individual functions / components in isolation
Integration	How modules interact with each other
System	Full application end-to-end flow
Acceptance (UAT)	Business requirements met from user perspective

Checklist

- [ ] All unit tests pass
- [ ] Integration tests pass
- [ ] UAT sign-off received
- [ ] No critical / high severity bugs open

🔄 Phase 5 — Pipeline & Deployment Setup

> ⚡ *Automate everything repeatable.*

`CI/CD Config → Deployment Strategy → Pre-deployment Validation → Deploy`

Step	Description
🔁 **CI/CD Pipeline**	Automate build, test, and deployment process
🗺️ **Deployment Strategy**	Define rollout rules (blue-green, canary, rolling)
✔️ **Pre-deployment Validation**	Ensure all gates pass before going live

Pipeline Flow

```
Code Push
   ↓
Trigger CI Pipeline
   ↓
Build → Lint → Test → Security Scan
   ↓
Artefact Created
   ↓
Pre-deploy Validation
   ↓
Deploy to Staging → Smoke Test
   ↓
Deploy to Production
   ↓
Post-deploy Health Check
```

Checklist

- [ ] CI pipeline runs on every PR
- [ ] Secrets managed via vault / environment variables
- [ ] Rollback plan documented
- [ ] Staging environment mirrors production

☁️ Phase 6 — Cloud Deployment Options

> 🌐 *Choose based on workload type, scale, and team expertise.*

Platform	Type	Best For
☁️ **Azure**	Full cloud platform	Enterprise workloads, Microsoft ecosystem
🖥️ **AWS EC2**	Virtual machines	Full-control server workloads
⚡ **AWS Lambda**	Serverless	Event-driven, short-lived functions
🐳 **AWS Fargate**	Managed containers	Containerised apps without managing servers

Decision Guide

```
Need full VM control?          → EC2
Running containers?            → Fargate
Event-driven / short tasks?    → Lambda
Microsoft / hybrid cloud?      → Azure
Need all of the above?         → Mix & match
```

Checklist

- [ ] Platform chosen and justified
- [ ] IAM roles and permissions configured
- [ ] Cost estimates reviewed
- [ ] Auto-scaling policies set

🏛️ Phase 7 — Platform Engineering

> 🔩 *The invisible foundation that keeps everything running.*

Area	Responsibility
🌍 **Infrastructure Management**	Manage cloud resources, IaC (Terraform / Pulumi)
🌐 **DNS Management**	Configure and maintain domain name systems
🏠 **Hosting Services**	Ensure reliable, scalable hosting environment
📶 **Service Reliability**	Maintain uptime SLAs and performance standards

Checklist

- [ ] Infrastructure defined as code (IaC)
- [ ] DNS records documented and version-controlled
- [ ] Uptime SLA defined (e.g. 99.9%)
- [ ] Runbooks written for common incidents

📡 Phase 8 — Monitoring & Observability

> 👁️ *You can't improve what you can't see.*

```
Deploy
  ↓
Web Monitoring + Traffic Analysis
  ↓
Load Balancing + CDN
  ↓
EC2 / Server Health Checks
  ↓
Pipeline Monitoring
  ↓
Alerting & Logging
  ↓
Feedback → Improve
```

Area	What to Track
🌐 **Web Monitoring**	Uptime, response time, availability
👤 **User Traffic**	Page views, sessions, behaviour patterns
⚖️ **Load Balancing**	Traffic distribution, server health
🚀 **CDN**	Cache hit rate, global latency
🖥️ **EC2 Instances**	CPU, memory, disk, network I/O
💓 **Server Health**	App errors, crash rates, response codes
🔁 **Pipeline Monitoring**	Build success rate, deploy frequency, MTTR
🚨 **Alerting & Logging**	On-call alerts, structured log aggregation

Key Metrics to Watch

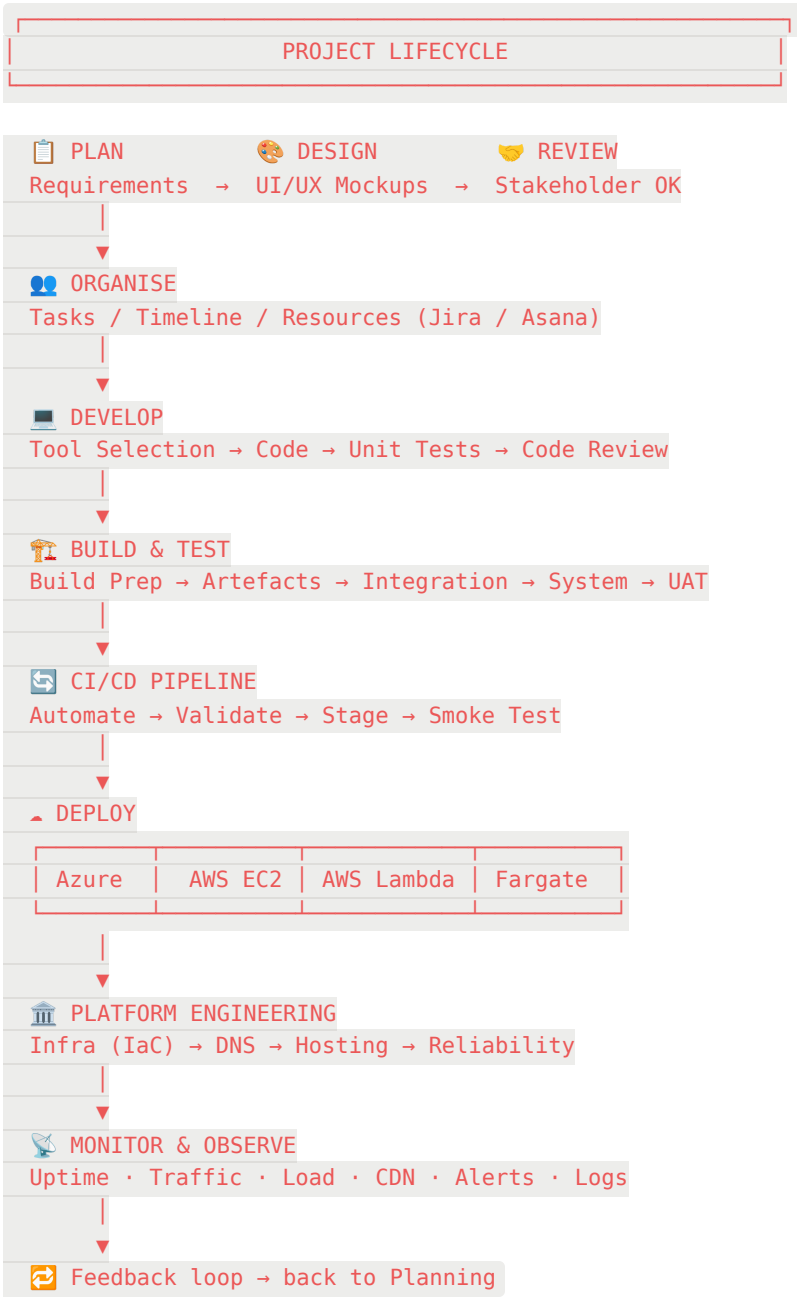
Metric	Target
Uptime	≥ 99.9%
Error rate	< 1%
P95 response time	< 500 ms
Deploy frequency	As high as stable
MTTR (Mean Time to Recover)	< 1 hour

Checklist

- [ ] Monitoring dashboards live (e.g. Datadog, Grafana, CloudWatch)
- [ ] Alerts configured for critical thresholds
- [ ] Logs centralised and searchable

- [ ] On-call rotation defined
- [ ] Post-incident review process in place

🔁 Full Flow Diagram



token is given , for each task the token is splited and given to the team / member , it comes with a validity , delay needs reason for TL.